Week 11 Level 3: Advanced

Project Studio 2

Test it, playtest it, polish it, explain it.

What you will do

- Write at least **five pytest tests** for your own classes (Milestone 3).
- **Playtest** with a partner and sort the bugs you find (**triage**).
- Write a **README** and check your project is **accessible** (Milestone 4).

Tests: crash-test dummies for your code

A test is a tiny program that checks your program. Test the **logic** (moving, scoring, saving), not the drawing. Test **normal** cases and **edge** cases: the corner, the wall, the empty list, the last life.

Example: tests for a tiny Player

muncher_demo.py

```python
class Player:
    def __init__(self, row, col):
        self.row = row
        self.col = col
        self.score = 0

    def move(self, grid, d_row, d_col):
        new_row, new_col = self.row + d_row, self.col + d_col
        if grid[new_row][new_col] == "#":
            return False
        self.row, self.col = new_row, new_col
        if grid[new_row][new_col] == "*":
            self.score += 10
            grid[new_row][new_col] = "."
        return True
```

test_muncher_demo.py

```python
from muncher_demo import Player

```

```
def make_grid():   # a tiny maze: wall, empty, star, empty, wall
    return [list("#####"), list("#.*.#"), list("#####")]

def test_player_moves_into_an_empty_cell():
    p = Player(1, 1)
    assert p.move(make_grid(), 0, 1) is True
    assert (p.row, p.col) == (1, 2)

def test_walls_block_the_player():          # edge case
    p = Player(1, 1)
    assert p.move(make_grid(), -1, 0) is False
    assert (p.row, p.col) == (1, 1)

def test_eating_a_star_scores_10_once():    # edge case
    grid = make_grid()
    p = Player(1, 1)
    p.move(grid, 0, 1)
    p.move(grid, 0, -1)
    p.move(grid, 0, 1)
    assert p.score == 10
```

How to run your tests: in Thonny choose **Tools** → **Open system shell...**, make sure you are in your project folder, and type `python -m pytest -q`. Each dot is a passing test; an F is a failing one, with its name and the line that failed.

Tip: Is my test any good?

Break your code on purpose. Does the test go red? If not, the test is not checking anything. Then fix the code and watch it go green.

Exercise 11.1: My five tests (Milestone 3)

Plan your tests, then write them in `test_<your project>.py`. At least one must be an edge case.

#	**Test name** (`test_...`)	**What it checks**	**Edge?**
1			
2			
3			
4			
5			

Playtest and triage

In a **playtest**, someone else uses your project while you watch **silently** and take notes. Every time they get stuck, you learn something. Afterwards, sort the notes into three piles – that is called **triage**:

1 Must fix	**2 Should fix**	**3 Nice to have**
crashes, cannot start, cannot win	confusing, unfair, hard to read	new ideas for version 2

For a crash: first write a test that shows the bug, then fix it. The test makes sure it never comes back.

Exercise 11.2: Playtest feedback cards

Cut these out (or copy them onto index cards). Fill in one card for each thing you notice while your partner plays.

What happened?

What did the player expect?

Pile: ☐ must ☐ should ☐ nice

What happened?

What did the player expect?

Pile: ☐ must ☐ should ☐ nice

What happened?

What did the player expect?

Pile: ☐ must ☐ should ☐ nice

What happened?

What did the player expect?

Pile: ☐ must ☐ should ☐ nice

README: explain it to a stranger

A README is the first thing people read. Future-you is a stranger too!

Exercise 11.3: My README (Milestone 4)

Create `README.md` in your project folder with these sections:

☐ **Title and one sentence**: what it is.

☐ **How to run it**: which file to open and press Run.

- ☐ **Controls**: keys or clicks.
- ☐ **How it works**: my classes (is-a / has-a), my data structure, my algorithm, the file it saves.
- ☐ **Tests**: how to run them, how many there are.
- ☐ **Credits**: my name as a nickname, and every borrowed thing with its license (or "everything made by me").

Exercise 11.4: Accessibility and usability check

- ☐ Colors come from the Okabe–Ito palette, and nothing depends on color alone (shapes, letters or text too).
- ☐ Text is big enough to read from a chair.
- ☐ The controls are written on the screen.
- ☐ Nothing flashes fast.
- ☐ A new player knows what to do within 10 seconds.
- ☐ One thing I fixed today: _____

Vocabulary box

edge case An unusual input at the limits: a corner, an empty list, zero, the last life.

playtest Watching someone use your project to find problems.

triage Sorting problems by how much they matter, to fix the important ones first.

README A file that explains what a project is, how to run it and how it works.

usability How easy something is to use for someone new.

accessibility Designing so that people with different abilities can use it too.

Reflection

What did your playtester do that surprised you?

Which test are you proudest of, and why?

Week 11 Level 3: Advanced

Project Studio 2

15–20 minutes. Finish Milestone 4 and get ready for the showcase.

Name (nickname is fine): _____ Date: _____

★ Core (everyone): Finish Milestone 4 and prepare your talk

1. Finish your `README.md`: title, how to run, controls, how it works (classes, data structure, algorithm, saved file), tests, credits. Commit it.
2. Fix one more "must fix" or "should fix" bug from your playtest cards. Commit it.
3. Run all your tests: `python -m pytest -q`. Number passing: _____ (goal: 5 or more)
4. Fill in the showcase talk card (workbook Exercise 12.1) and practice once out loud with a timer: 2 minutes.

★★ Challenge (want more?): An edge-case hunt

Think like a sneaky player: what could break your project? (Pressing keys very fast, walking into a corner, an empty save file, the last life.) Write a test for one of them. If it fails, fix the code. Commit with a message like `Test and fix empty high-score file`.

Creative (make it yours): Box art

Design the box or poster for your project as if it were sold in a shop: the title, one exciting sentence, a picture, and "Made by" with your *nickname*. Keep it at home – no posting online.

Self-check: tick each box when it is true

☐ I have 5 or more passing tests, including an edge case.

☐ My README explains my classes, data structure and algorithm.

- ☐ My project passes the accessibility checklist (not color alone, readable text, controls on screen).
- ☐ I have 6 or more commits and know my talk's Problem, Choice, Why and Next time.